Administración de Sistemas Gestores de Bases de Datos: MySQL

Apuntes teórico-prácticos sobre la Tríada CID, gestión de usuarios, roles, permisos, esquemas y seguridad en MySQL.

La Tríada CID (Confidencialidad, Integridad y Disponibilidad) en MySQL

La tríada CID es un modelo de seguridad fundamental para garantizar la protección de la información en cualquier sistema informático, incluido MySQL. Este modelo se utiliza para establecer las bases de las políticas de seguridad en el SGBD.

1. Confidencialidad (Confidentiality)

Garantiza que la información solo sea accesible para usuarios y procesos autorizados. En MySQL se logra mediante:

- **Autenticación y Control de Acceso:** MySQL valida las credenciales mediante el par `'usuario'@'host'` utilizando componentes modernos de autenticación como `caching_sha2_password`.
- **Privilegios y Roles:** Control granular mediante privilegios globales, de base de datos, de tabla o de columna, además del uso de roles para agrupar permisos.
- **Cifrado de datos (Encryption):** Soporte de **TDE (Transparent Data Encryption)** para cifrar tablas y tablespaces en reposo, y conexiones cifradas mediante **TLS/SSL** para datos en tránsito.
- **Auditoría:** Plugin de auditoría (MySQL Audit Plugin) para registrar operaciones, inicios de sesión y accesos no autorizados a datos sensibles.

2. Integridad (Integrity)

Asegura que los datos se mantengan exactos, coherentes y no sean modificados de forma indebida o accidental.

- **Restricciones (Constraints):** Definición de claves primarias (`PRIMARY KEY`), claves foráneas (`FOREIGN KEY`), unicidad (`UNIQUE`), valores no nulos (`NOT NULL`) y validaciones personalizadas (`CHECK`).

- **Validaciones y Triggers:** Disparadores (`TRIGGERS`) e instrucciones almacenadas para validar reglas de negocio complejas antes de insertar o modificar datos.
- **Cumplimiento ACID y Control de Transacciones:** El motor InnoDB garantiza la integridad mediante transacciones que cumplen las propiedades ACID (Atomisidad, Consistencia, Aislamiento y Durabilidad).
- **Copias de seguridad y Binlog:** Copias periódicas y uso del registro binario (*Binary Log*) para garantizar la recuperación fiable ante fallos.

3. Disponibilidad (Availability)

Garantiza que la base de datos se mantenga operativa y accesible cuando los usuarios o aplicaciones autorizados lo requieran.

- **Alta Disponibilidad y Replicación:** Soluciones como *MySQL InnoDB Cluster*, *MySQL Group Replication* o arquitectura Primario-Secundario (Master-Slave) para tolerancia a fallos.
- **Recuperación ante desastres:** Réplicas asíncronas o semisíncronas en ubicaciones secundarias.
- **Copias de seguridad y restauración:** Herramientas como `mysqldump` , `mysqlpump` o *MySQL Enterprise Backup* para la restauración rápida del servicio.
- **Monitorización:** Herramientas como *MySQL Enterprise Monitor* o `Performance Schema` para analizar la salud del sistema y evitar paradas no planificadas.

> 💡 **Resumen de la Tríada CID**
>
> - **Confidencialidad:** Autenticación por host, cifrado SSL/TDE, permisos y roles.
> - **Integridad:** Restricciones relacionales, motor transaccional InnoDB y triggers.
> - **Disponibilidad:** Replicación, InnoDB Cluster, backups y binlogs.

Gestión de Usuarios en MySQL

En MySQL, la identidad de un usuario se define por la combinación de un nombre de usuario y el host desde el cual se conecta: `'usuario'@'host'` .

🔑 Formato de Identificación y Autenticación

A diferencia de otros SGBD, MySQL distingue el origen de la conexión:

- `'alumne'@'localhost'` : Solo permite conexiones desde la propia máquina local.

- `'alumne'@'192.168.1.50'` : Solo permite conexiones desde una IP específica.
- `'alumne'@'%'` : Permite conexiones desde cualquier host remoto (comodín `%`).

El método de autenticación por defecto en MySQL 8.0+ es `caching_sha2_password` , ofreciendo un cifrado seguro de credenciales.

🛠️ Creación, Modificación y Eliminación de Usuarios

Las operaciones de gestión de usuarios requieren el privilegio global `CREATE USER` o ser superusuario (`GRANT OPTION` / `SYSTEM_USER`).

```
-- Crear un usuario normal (conecta solo desde localhost)
CREATE USER 'alumne'@'localhost' IDENTIFIED BY '1234';

-- Crear un usuario que puede conectar desde cualquier IP (%)
CREATE USER 'profe'@'%' IDENTIFIED BY 'abc123';

-- Modificar contraseña de un usuario
ALTER USER 'alumne'@'localhost' IDENTIFIED BY 'NuevaPass2026!';

-- Forzar el plugin de autenticación si fuera necesario
ALTER USER 'alumne'@'localhost' IDENTIFIED WITH caching_sha2_password BY 'NuevaPass2026!';

-- Eliminar un usuario
DROP USER 'alumne'@'localhost';
```

> ⚠️ **Importante al eliminar usuarios en MySQL**
>
> En MySQL, si eliminas un usuario con `DROP USER` , sus conexiones activas no se cortan automáticamente. Además, a diferencia de PostgreSQL, eliminar un usuario no falla si posee tablas u objetos, ya que los objetos pertenecen a las bases de datos (esquemas) y no al usuario como propietario individual.

🔍 Consultas útiles para la gestión de usuarios

```
-- Ver todos los usuarios existentes y su plugin de autenticación
SELECT User, Host, plugin, account_locked, password_expired FROM mysql.user;

-- Ver la sesión y el usuario actual
SELECT USER(), CURRENT_USER();
-- USER(): Muestra el usuario y host con el que te intentaste conectar.
```

```
-- CURRENT_USER(): Muestra la cuenta con la que MySQL ha validado los permisos en la sesión actual.

-- Ver el nombre de la base de datos/esquema actual
SELECT DATABASE();
```

Esquemas (Schemas) y Bases de Datos en MySQL

En MySQL, a diferencia de PostgreSQL u Oracle, **DATABASE y SCHEMA son exactamente lo mismo**. Los términos se usan indistintamente como sinónimos.

```
CREATE DATABASE colegio;
-- Es 100% equivalente a:
CREATE SCHEMA colegio;
```

📌 Concepto de Esquema/Base de Datos

Un esquema en MySQL es un contenedor lógico de objetos (tablas, vistas, procedimientos, funciones, disparadores). No existe el concepto de "esquema public" por defecto como en PostgreSQL.

```
-- Seleccionar la base de datos sobre la que trabajar
USE colegio;

-- Crear una tabla dentro de la base de datos seleccionada
CREATE TABLE alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);

-- Hacer referencia a una tabla de otra base de datos mediante sintaxis explícita:
SELECT * FROM colegio.alumnos;
```

> 💡 **Equivalencia entre SGBDs**
>
> En PostgreSQL o Oracle existe una jerarquía (Instancia → Base de datos → Esquema → Tabla). En MySQL, la estructura es directa: **Servidor (Instancia) → Base de Datos / Esquema → Tabla**.

Bloqueo de Usuarios, Caducidad y Gestión de Sesiones

🔐 Bloquear y Desbloquear Cuentas

En MySQL es posible bloquear de manera nativa la cuenta de un usuario sin necesidad de cambiar su contraseña ni eliminarlo:

```
-- Bloquear un usuario (impide iniciar sesión)
ALTER USER 'omar'@'localhost' ACCOUNT LOCK;

-- Desbloquear cuenta
ALTER USER 'omar'@'localhost' ACCOUNT UNLOCK;
```

⏳ Caducidad y Políticas de Contraseñas

MySQL permite configurar políticas de expiración de contraseñas de forma global o individual:

```
-- Establecer que la contraseña caduque en una fecha específica
ALTER USER 'omar'@'localhost' PASSWORD EXPIRE; -- Caduca inmediatamente

-- Configurar que expire cada 90 días
ALTER USER 'omar'@'localhost' PASSWORD EXPIRE INTERVAL 90 DAY;

-- Restablecer a la política por defecto
ALTER USER 'omar'@'localhost' PASSWORD EXPIRE DEFAULT;

-- Forzar que la contraseña NUNCA caduque
ALTER USER 'omar'@'localhost' PASSWORD EXPIRE NEVER;
```

> 💡 **Forzar cambio de contraseña en el próximo inicio de sesión**
>
> Con `PASSWORD EXPIRE`, la próxima vez que el usuario conecte se le permitirá autenticarse, pero MySQL restringirá la ejecución de cualquier consulta hasta que ejecute un `ALTER USER` para cambiar su clave.

🔌 Gestión y Desconexión de Sesiones Activas

Para consultar las sesiones de usuarios conectados actualmente:

```
-- Ver procesos y conexiones activas
SHOW PROCESSLIST;
-- O consultando la tabla de rendimiento:
SELECT ID, USER, HOST, DB, COMMAND, TIME, STATE, INFO FROM information_schema.processlist;
```

Para cancelar una consulta o cerrar la sesión de un usuario de forma inmediata:

```
-- Cancelar la consulta activa sin cerrar la conexión
KILL QUERY 105; -- (105 es el ID del hilo/proceso)

-- Terminar la sesión completamente (desconectar al usuario)
KILL CONNECTION 105;
```

Sistema de Permisos (Privilegios) en MySQL

Un privilegio en MySQL es un derecho específico que se otorga a un usuario o rol para ejecutar comandos SQL o gestionar componentes del servidor.

📋 Niveles de Privilegios en MySQL

1. **Nivel Global:** Se aplica a todas las bases de datos del servidor (`*.*`). Ejemplo: `CREATE USER` , `SHUTDOWN` , `PROCESS` .
2. **Nivel de Base de Datos:** Se aplica a todos los objetos dentro de un esquema (`nombre_bd.*`). Ejemplo: `CREATE` , `DROP` , `ALTER` .
3. **Nivel de Tabla:** Se aplica a una tabla específica (`nombre_bd.nombre_tabla`). Ejemplo: `SELECT` , `INSERT` , `UPDATE` , `DELETE` .
4. **Nivel de Columna:** Se limita a columnas concretas de una tabla. Ejemplo: `SELECT (nombre, telefono)` , `UPDATE (direccion)` .
5. **Nivel de Rutina:** Se aplica a procedimientos almacenados y funciones (`EXECUTE`).

🛠️ Concesión y Revocación de Privilegios (GRANT / REVOKE)

```
-- Conceder conexión y lectura completa sobre una base de datos
GRANT SELECT ON colegio.* TO 'alumne'@'localhost';

-- Conceder permisos DML sobre todas las tablas de un esquema
GRANT SELECT, INSERT, UPDATE, DELETE ON colegio.* TO 'profesor'@'%';
```

```
-- Privilegios a nivel de columna
GRANT SELECT (nombre, telefono), UPDATE (telefono) ON colegio.alumnos TO 'recepcion'@'localhost';

-- Activar la propagación de permisos (WITH GRANT OPTION)
GRANT SELECT ON colegio.* TO 'jefe_estudios'@'%' WITH GRANT OPTION;

-- Revocar permisos específicos
REVOKE DELETE ON colegio.* FROM 'profesor'@'%';

-- Aplicar los cambios en privilegios inmediatos (recomendado tras modificar tablas del sistema)
FLUSH PRIVILEGES;
```

🔍 Consultar Privilegios Otorgados

```
-- Ver los privilegios de un usuario específico
SHOW GRANTS FOR 'alumne'@'localhost';

-- Ver privilegios propios
SHOW GRANTS FOR CURRENT_USER();
```

> ⚠️ **Comportamiento de UPDATE/DELETE sin SELECT**
>
> Si un usuario tiene permisos de `UPDATE` o `DELETE` pero carece de permiso `SELECT` sobre las columnas utilizadas en la cláusula `WHERE`, la sentencia fallará por falta de privilegios de lectura para evaluar la condición.

Gestión de Roles en MySQL

A partir de **MySQL 8.0** se incluye el soporte nativo para **Roles**. Un rol es un contenedor nombrado que agrupa un conjunto de privilegios de seguridad.

🛠️ Creación, Asignación y Activación de Roles

```
-- 1. Crear un rol
CREATE ROLE 'rol_desarrollador', 'rol_consultor';

-- 2. Asignar privilegios al rol
GRANT SELECT, INSERT, UPDATE, DELETE ON colegio.* TO 'rol_desarrollador';
```

```
GRANT SELECT ON colegio.* TO 'rol_consultor';

-- 3. Asignar el rol a un usuario
GRANT 'rol_desarrollador' TO 'desarrollador1'@'localhost';

-- 4. Revocar un rol
REVOKE 'rol_desarrollador' FROM 'desarrollador1'@'localhost';
```

> ⚠️ **Activación de Roles en MySQL (Muy Importante)**
>
> Por defecto en MySQL, cuando a un usuario se le concede un rol, **el rol no está activo automáticamente** cuando inicia sesión. El usuario debe activarlo o la sesión debe configurarse para activarlos por defecto.
>
> ```
> -- Activar los roles asignados en la sesión actual:
> SET ROLE 'rol_desarrollador';
> -- O activar todos los roles concedidos:
> SET ROLE ALL;
>
> -- Configurar para que MySQL active TODOS los roles de forma automática al iniciar sesión:
> SET GLOBAL activate_all_roles_on_login = ON;
> -- O de forma específica para un usuario:
> ALTER USER 'desarrollador1'@'localhost' DEFAULT ROLE ALL;
> ```

Vistas como Elemento de Seguridad en MySQL

Las vistas son tablas virtuales basadas en una consulta SQL. Constituyen un mecanismo fundamental para controlar la visibilidad y proteger la información confidencial.

1. Restricción de Acceso (Vistas Horizontales, Verticales y Mixtas)

```
-- Vista Vertical (Oculta columnas sensibles como salarios o passwords)
CREATE VIEW vista_empleados_publica AS
SELECT id, nombre, departamento FROM empleados;

-- Vista Horizontal (Filtra filas según una condición)
CREATE VIEW vista_empleados_madrid AS
SELECT * FROM empleados WHERE ciudad = 'Madrid';
```

```
-- Vista Mixta (Filtra columnas y filas)
CREATE VIEW vista_empleados_comercial AS
SELECT id, nombre, telefono FROM empleados WHERE departamento = 'Ventas';
```

2. Control de Acceso Mediante Vistas

Podemos denegar el acceso a la tabla base a un usuario y concederle únicamente permisos sobre la vista:

```
-- Revocar o no conceder permisos sobre la tabla principal
-- Conceder acceso únicamente a la vista
GRANT SELECT ON colegio.vista_empleados_publica TO 'usuario_consulta'@'localhost';
```

3. Cláusula SQL SECURITY (DEFINER vs INVOKER)

MySQL permite definir el contexto de seguridad bajo el cual se ejecutará la vista:

- `SQL SECURITY DEFINER` (Por defecto): La vista se ejecuta con los permisos del usuario que la **creó**. Permite dar acceso a datos filtrados de tablas a las que el usuario que consulta no tiene acceso directo.
- `SQL SECURITY INVOKER` : La vista se ejecuta con los permisos del usuario que **ejecuta la consulta**. Si este usuario no tiene acceso a las tablas base, la consulta fallará.

```
-- Crear vista ejecutada con privilegios del creador
CREATE DEFINER = 'admin'@'localhost' SQL SECURITY DEFINER VIEW vista_protegida AS
SELECT nombre, email FROM usuarios_sistema;
```

4. Vistas Actualizables y la opción WITH CHECK OPTION

Una vista es actualizable si es simple (basada en una sola tabla, sin agregaciones, `GROUP BY` , `DISTINCT` o `UNION`).

La cláusula `WITH CHECK OPTION` evita que se inserten o modifiquen filas mediante la vista si no cumplen con las condiciones fijadas en la definición de la vista.

```
CREATE VIEW vista_alumnos_aprobados AS
SELECT id, nombre, nota FROM alumnos WHERE nota >= 5
WITH CHECK OPTION;

-- Si intentas insertar un alumno con nota 4 a través de la vista, MySQL rechazará la operación.
```